

# ผลการทดลอง stage 2 — ตัดฟีเจอร์ระดับไหนดีที่สุด

## การทดลอง

7 ระดับการตัดฟีเจอร์ × 3 แบบการแบ่งข้อมูล × 3 โมเดล = 63 การเทรน

## 7 ระดับ (ตัดสะสมกันไปเรื่อย ๆ)

| ระดับ        | ตัดเพิ่มจากระดับก่อน                                       | #feat |
|--------------|------------------------------------------------------------|-------|
| raw          | — (identifier ถูก encode เป็นเลข ไม่ได้ตัด)                | 83    |
| no_id        | Flow ID Src IP Dst IP Timestamp                            | 79    |
| no_const     | 12 คอลัมน์ค่าคงที่ (บีก CICFlowMeter ให้ 0 ทั้งหมด)        | 67    |
| no_port      | Src Port Dst Port                                          | 65    |
| no_dup       | 8 คอลัมน์ที่ correlation = 1.0000                          | 57    |
| no_expensive | Active/Idle × Mean, Std, Max, Min (คำนวณ real-time ไม่ได้) | 49    |
| shap_top15   | SHAP คัดจากระดับ no_port                                   | 15    |

## 3 แบบการแบ่งข้อมูล

| split    | train                    | test                     | บอกอะไร                              |
|----------|--------------------------|--------------------------|--------------------------------------|
| random   | 80% ปั่นกันหมด           | 20%                      | ทำข้อสอบชุดเดิม — จับ leakage ไม่ได้ |
| ovs2meta | Normal½ + OVS            | Normal½ + metasploitable | ทำข้อสอบชุดใหม่                      |
| meta2ovs | Normal½ + metasploitable | Normal½ + OVS            | ทำข้อสอบชุดใหม่ (ทิศกลับ)            |

ทำไมต้องมี 3 แบบ: random อย่างเดียวจับการ "จำ" ไม่ได้ เพราะ IP เดียวกัน อยู่ทั้ง train และ test — โมเดลที่จำ IP ได้ก็ยังทายถูก ต้องข้าม testbed ถึงจะรู้ว่าเรียนพฤติกรรมจริงหรือแค่ท่องจำ

## Hyperparameter

จูนด้วย **Random Search 30 settings + 3-fold CV** บน train set เท่านั้น ช่วงค่าอ้างอิงจาก Table I ของงานที่ benchmark บน InSDN เดียวกัน (2022 ASYU — GBDT + SHAP on CIC-IDS2017 / CSE-CIC-IDS2018 / InSDN)

**max\_depth** จำกัดที่ 6-12 เท่ากันทุกโมเดล สำคัญมาก เพราะ default ของ sklearn คือไม่จำกัด ส่วน XGBoost คือ 6 ถ้าไม่คุมจะเทียบกันไม่ยุติธรรม

**notebook** นี้อ่านผลจาก **outputs/\*.csv** อย่างเดียว ไม่คำนวณอะไรใหม่ → ตัวเลขตรงกับที่ **.py** ผลิตเสมอ

```
In [1]: import pandas as pd
import matplotlib.pyplot as plt

import config

plt.rcParams['font.family'] = ['Thonburi', 'DejaVu Sans']

grid = pd.read_csv(config.OUT_DIR / '02_stage2_grid.csv')
print(f"{len(grid)} แถว = {grid.level.nunique()} ระดับ × "
      f"{grid.split.nunique()} split × {grid.model.nunique()} โมเดล")
```

63 แถว = 7 ระดับ × 3 split × 3 โมเดล

## แต่ละระดับตัดอะไร และตัดเพราะอะไร

ตัด สละกันไปเรื่อย ๆ ระดับหลังตัดทุกอย่างของระดับก่อน บวกกลุ่มใหม่ จะได้แยกออกว่ากลุ่มไหนมีผลแค่ไหน

### 1. raw (83) ไม่ตัดอะไรเลย

identifier ถูก **encode** เป็นเลข ไม่ได้ตัดทิ้ง เพื่อให้ทดลองได้ว่า "ถ้าไม่ตัดจะเป็นยังไง" ไม่จำเป็นต้องคำตอบนี้ไม่ได้เลย

### 2. no\_id (79) ตัด identifier 4 ตัว

| คอลัมน์   | ค่าไม่ซ้ำ     | เหตุผล                                                    |
|-----------|---------------|-----------------------------------------------------------|
| Flow ID   | 234,971 (68%) | string จาก IP+Port+Protocol เกือบเป็นเลขประจำแถว          |
| Src IP    | 122,868 (36%) | สูงเพราะ DDoS ปลอม source IP · เครื่องที่ยิงใช้ IP ตายตัว |
| Dst IP    | 1,081         | เครื่องเหยื่อใช้ IP เดิมตลอด                              |
| Timestamp | 1,392         | attack ยิ่งเป็นช่วงเวลา โมเดลจำ "ช่วงนี้คือ attack"       |

ไม่ได้ตัดเพราะรั่วประโยชน์ แต่ตัดเพราะมีประโยชน์เกินไปแบบผิด ๆ โมเดลจะจำ IP แทนที่จะเรียนพฤติกรรม ใช้กับเครือข่ายจริงไม่ได้

หลักฐาน: SHAP ที่ระดับ raw จัด Timestamp เป็นอันดับ 1 และ Flow ID อันดับ 2 (identifier กิน 3 ใน 5 อันดับแรก) และ macro-F1 ช้าม testbed อยู่ที่ 0.07

### 3. no\_const (67) ตัดคอลัมน์ค่าคงที่ 12 ตัว

บ๊ักของ CICFlowMeter คอลัมน์กลุ่ม bulk ออกมาเป็น 0 ทั้งคอลัมน์

ทำไมต้องตัดก่อนใช้ SHAP: ฟีเจอร์ค่าเดียวจะได้ SHAP = 0 แล้วเราจะสรุปว่า "ไม่สำคัญ" ซึ่งผิด มันไม่สำคัญเพราะ *ไม่มีข้อมูล* ไม่ใช่เพราะโมเดลพิจารณาแล้วไม่ใช่

และถ้าไม่ตัดจะเคลมเกินจริง ว่า SHAP ลดฟีเจอร์ได้ 83→15 ทั้งที่ 12 ตัวนั้นตัดได้ด้วยโค้ดบรรทัดเดียว ไม่ต้องใช้ SHAP

### 4. no\_port (65) ตัด Src Port Dst Port

ใน testbed port ผูกกับชนิด attack เกือบ 1:1 (BFA→22 · Web-Attack→80) โมเดลจะจำ port แทนพฤติกรรม พอเจอ brute-force ที่ port 2222 ก็พลาด

พิสูจน์แล้ว: พอตัด identifier ออก Dst Port ได้จากอันดับ 7 ขึ้นมาเป็น อันดับ 3 แล้วพอตัดคอลัมน์ค่าคงที่อีกชั้น ขึ้นเป็น อันดับ 1 → โมเดลไม่ได้หันไปเรียนพฤติกรรม แต่ทำ shortcut ตัวถัดไป

### 5. no\_dup (57) ตัดคอลัมน์ซ้ำ 8 ตัว

คู่ที่ correlation = 1.0000 (ข้อมูลเดียวกันคนละชื่อ) เก็บชื่อมาตรฐานกว่าไว้

| เก็บ             | ตัด              |
|------------------|------------------|
| Tot Fwd Pkts     | Subflow Fwd Pkts |
| Tot Bwd Pkts     | Subflow Bwd Pkts |
| TotLen Fwd Pkts  | Subflow Fwd Byts |
| TotLen Bwd Pkts  | Subflow Bwd Byts |
| Fwd Pkt Len Mean | Fwd Seg Size Avg |
| Bwd Pkt Len Mean | Bwd Seg Size Avg |
| PSH Flag Cnt     | Bwd PSH Flags    |
| URG Flag Cnt     | Bwd URG Flags    |

กระทาบ SHAP โดยตรง: ฟีเจอร์ซ้ำกัน 2 ตัวจะแบ่งความสำคัญกันคนละครึ่ง ทำให้ทั้งคู่ดูไม่สำคัญ ทั้งที่จริงสำคัญมาก อาจตกกรอบ Top-K ทั้งคู่

## 6. no\_expensive (49) ตัด Active/Idle × Mean, Std, Max, Min

ฟีเจอร์กลุ่มนี้ ต้องรอ flow จบก่อนถึงคำนวณได้ ใช้กรอง real-time ไม่ได้

เป็นข้ออ้างหลักของการลดฟีเจอร์ที่เราเขียนไว้ตั้งแต่ต้น แต่ไม่เคยพิสูจน์ ระดับนี้จึงมีไว้ทดสอบว่าตัดแล้วเสียอะไรจริงไหม

## 7. shap\_top15 (15) SHAP คัด

เอา SHAP จากระดับ no\_port มาเฉลี่ยข้ามคลาส แล้วตัด Top-15 นี่คือระดับเดียวที่ใช้ XAI ตัดสินใจ ระดับ 1-6 ตัดด้วยเหตุผลเชิงโครงสร้าง

```
In [2]: # รายชื่อจริงที่ 01 คำนวณไว้ (ไม่ใช่ hardcoded)
import json

drops = json.loads((config.OUT_DIR / '01_drop_lists.json').read_text(encoding='utf-8'))
for group, cols in drops.items():
    print(f"\n{group} ({len(cols)} คอลัมน์)")
    for c in cols:
        print(f"    {c}")
```

```

id    (4 คอลัมน์)
      Flow ID
      Src IP
      Dst IP
      Timestamp

port  (2 คอลัมน์)
      Src Port
      Dst Port

const (12 คอลัมน์)
      Fwd PSH Flags
      Fwd URG Flags
      CWE Flag Count
      ECE Flag Cnt
      Fwd Byts/b Avg
      Fwd Pkts/b Avg
      Fwd Blk Rate Avg
      Bwd Byts/b Avg
      Bwd Pkts/b Avg
      Bwd Blk Rate Avg
      Init Fwd Win Byts
      Fwd Seg Size Min

dup   (8 คอลัมน์)
      Subflow Fwd Pkts
      Subflow Bwd Pkts
      Subflow Fwd Byts
      Subflow Bwd Byts
      Fwd Seg Size Avg
      Bwd Seg Size Avg
      Bwd PSH Flags
      Bwd URG Flags

expensive (8 คอลัมน์)
      Active Mean
      Active Std
      Active Max
      Active Min
      Idle Mean
      Idle Std
      Idle Max
      Idle Min

```

---

## ตารางหลัก — 7 ระดับ × 9 คอลัมน์

ค่าในตารางคือ **macro-F1** (เฉลี่ยแบบให้ทุกคลาสน้ำหนักเท่ากัน)

ใช้ macro ไม่ใช่ accuracy เพราะข้อมูล imbalance 7,173 เท่า (DDoS 121,942 แถว · U2R 17 แถว) — accuracy จะถูกคลาสใหญ่กลบ

```
In [3]: MODELS = ['DecisionTree', 'RandomForest', 'XGBoost']
SHORT = {'DecisionTree': 'DT', 'RandomForest': 'RF', 'XGBoost': 'XGB'}

main = grid.pivot_table(index='level', columns=['split', 'model'], values='f
main = main.reindex(config.FEATURE_LEVELS)
main = main[[s, m] for s in config.SPLITS for m in MODELS]]
main.columns = pd.MultiIndex.from_tuples(
    [(s, SHORT[m]) for s, m in main.columns])

n_feat = grid.groupby('level')['n_features'].first().reindex(config.FEATURE_
main.insert(0, ('', '#feat'), n_feat.astype(int))

# ไฮไลต์ตัวที่ชนะในแต่ละ (ระดับ × split)
# เขียนรวม = ชนะ · เขียนอ่อน = ต่างจากผู้ชนะไม่เกิน 0.005 (ถือว่าเสมอ)
def highlight_winner(df):
    st = pd.DataFrame('', index=df.index, columns=df.columns)
    for s in config.SPLITS:
        cols = [c for c in df.columns if c[0] == s]
        for i in df.index:
            row = df.loc[i, cols]
            best = row.max()
            for col in cols:
                if row[col] == best:
                    st.loc[i, col] = 'background-color:#2d9c5a;color:white;f
                elif best - row[col] <= 0.005:
                    st.loc[i, col] = 'background-color:#c7ecd4'

    return st

display(main.round(4).style.apply(highlight_winner, axis=None))
```

|              | #feat | DT       | RF       | random   |          | ovs2meta |          |
|--------------|-------|----------|----------|----------|----------|----------|----------|
|              |       |          |          | XGB      | DT       | RF       | XGB      |
| level        |       |          |          |          |          |          |          |
| raw          | 83    | 0.952000 | 0.981900 | 0.976800 | 0.350300 | 0.441600 | 0.375300 |
| no_id        | 79    | 0.855800 | 0.963900 | 0.980200 | 0.455900 | 0.470400 | 0.480800 |
| no_const     | 67    | 0.945500 | 0.967900 | 0.992200 | 0.492100 | 0.482200 | 0.480500 |
| no_port      | 65    | 0.831800 | 0.919400 | 0.968400 | 0.382300 | 0.471900 | 0.463100 |
| no_dup       | 57    | 0.873200 | 0.921400 | 0.939900 | 0.467700 | 0.470300 | 0.458800 |
| no_expensive | 49    | 0.907500 | 0.923100 | 0.927100 | 0.455500 | 0.471900 | 0.463400 |
| shap_top15   | 15    | 0.819200 | 0.948200 | 0.977500 | 0.547100 | 0.481600 | 0.459900 |

## วิธีอ่านตาราง

อ่านตามแนวนอน — ระดับเดียวกัน เทียบว่าโมเดลไหนดีกว่า

อ่านตามแนวตั้ง — โมเดลเดียวกัน เทียบว่าตัดฟิเจอร์แล้วดีขึ้น/แย่ลง

เทียบข้ามกลุ่ม **split** ← สำคัญที่สุด ดูว่าค่าใน **random** กับ **meta2ovs** ห่างกันแค่ไหน → ห่างมาก = โมเดลจำ testbed มากกว่าเรียนรู้พฤติกรรม

## โมเดลไหนชนะก็ครั้ง

```
In [4]: # นับว่าโมเดลไหนชนะก็ครั้ง แยกตาม split
wins = []
for s in config.SPLITS:
    sub = grid[grid.split == s]
    for lv in config.FEATURE_LEVELS:
        d = sub[sub.level == lv]
        if d.empty:
            continue
        wins.append({'split': s, 'level': lv,
                    'winner': SHORT[d.loc[d.f1.idxmax(), 'model']]})

w = pd.DataFrame(wins)
tally = w.pivot_table(index='winner', columns='split', aggfunc='size', fill_
tally['รวม'] = tally.sum(axis=1)
display(tally.sort_values('รวม', ascending=False))

print('จำนวนครั้งที่ชนะ (จาก 7 ระดับ ต่อ split)')
print('ระวัง: ชนะบน random อย่างเดียวไม่พอ - raw ชนะเพราะจำ ไม่ใช่เก่ง')
```

| split  | meta2ovs | ovs2meta | random | รวม |
|--------|----------|----------|--------|-----|
| winner |          |          |        |     |
| XGB    | 3        | 1        | 6      | 10  |
| RF     | 4        | 4        | 1      | 9   |
| DT     | 0        | 2        | 0      | 2   |

จำนวนครั้งที่ชนะ (จาก 7 ระดับ ต่อ split)  
ระวัง: ชนะบน random อย่างเดียวไม่พอ - raw ชนะเพราะจำ ไม่ใช่เก่ง

---

## ช่องว่าง random - cross-env = ขนาดของการ "จำ"

ถ้าโมเดลเรียนรู้พฤติกรรมจริง ผลบนข้อสอบใหม่ควรใกล้เคียงข้อสอบเดิม ถ้าโมเดลจำ testbed ผลจะว่างแรงเมื่อเปลี่ยน testbed

```
In [5]: g = grid.pivot_table(index=['level', 'model'], columns='split', values='f1')
g['cross_env_avg'] = g[['ovs2meta', 'meta2ovs']].mean(axis=1)
g['gap'] = g['random'] - g['cross_env_avg']
```

```
gap = g['gap'].unstack()[MODELS].reindex(config.FEATURE_LEVELS)
gap.columns = [SHORT[m] for m in gap.columns]
gap.insert(0, '#feat', n_feat.astype(int))

display(gap.round(4).style.background_gradient(
    subset=['DT', 'RF', 'XGB'], cmap='Reds'))
```

|              | #feat | DT       | RF       | XGB      |
|--------------|-------|----------|----------|----------|
| level        |       |          |          |          |
| raw          | 83    | 0.633600 | 0.616800 | 0.754800 |
| no_id        | 79    | 0.521900 | 0.517800 | 0.508400 |
| no_const     | 67    | 0.483800 | 0.523700 | 0.525800 |
| no_port      | 65    | 0.431000 | 0.438500 | 0.510000 |
| no_dup       | 57    | 0.538100 | 0.450500 | 0.525400 |
| no_expensive | 49    | 0.579900 | 0.453400 | 0.464000 |
| shap_top15   | 15    | 0.410200 | 0.478500 | 0.510400 |

สีเข้ม = ช่องว่างกว้าง = จำนวนกว่าเรียน

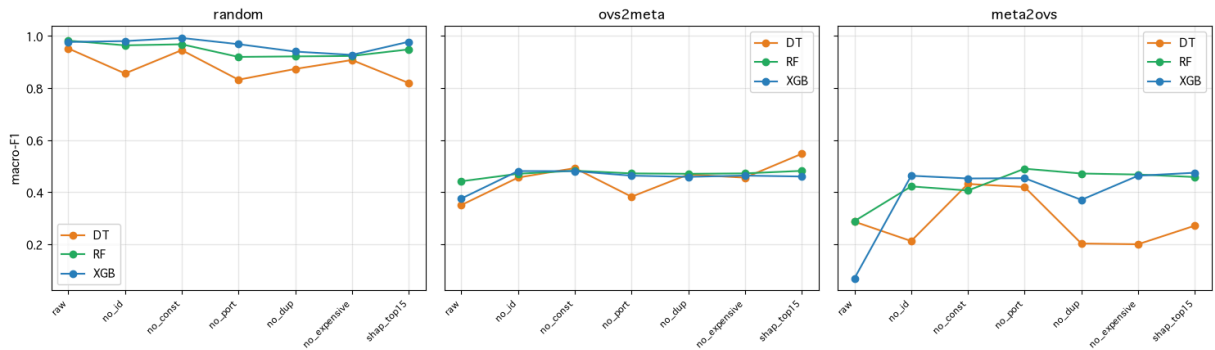
⚠ ข้อควรระวัง: ช่องว่างแคบไม่ได้แปลว่าดีเสมอไป ถ้าโมเดลแย่งทั้งสองฝั่ง ช่องว่างก็แคบ — ต้องดูคู่กับตารางหลักด้านบน

```
In [6]: fig, axes = plt.subplots(1, 3, figsize=(15, 4.5), sharey=True)
        colors = {'DecisionTree': '#e67e22', 'RandomForest': '#27ae60', 'XGBoost': '#34495e'}

        for ax, split in zip(axes, config.SPLITS):
            sub = grid[grid.split == split]
            for m in MODELS:
                d = sub[sub.model == m].set_index('level').reindex(config.FEATURE_LEVELS)
                ax.plot(range(len(d)), d['f1'], marker='o', label=SHORT[m], color=colors[m])
            ax.set_xticks(range(len(config.FEATURE_LEVELS)))
            ax.set_xticklabels(config.FEATURE_LEVELS, rotation=45, ha='right', fontsize=10)
            ax.set_title(split)
            ax.grid(alpha=.3)
            ax.legend()

        axes[0].set_ylabel('macro-F1')
        plt.tight_layout()
        plt.savefig(config.FIG_DIR / 'stage2_levels.png', dpi=150, bbox_inches='tight')
        plt.show()
```





## สิ่งที่ตารางบอก

### 1. raw + DecisionTree = ลายเซ็นของการจำ

random 0.9819 → meta2ovs 0.0679 ช่องว่าง 0.914

ทำข้อสอบชุดเดิมได้เกือบเต็ม แต่พอเปลี่ยน testbed พังสิ้นเชิง

นี่คือหลักฐานที่ตอบคำถาม "รู้ได้ไงว่าถ้าไม่ตัด identifier มันจะผิด" ได้ตรงที่สุด

### 2. ตัด constant + dup = ได้ฟรี

no\_id (79) = no\_const (67)      ทุกช่องเท่ากันเป๊ะ  
no\_port (65) = no\_dup (57)      ทุกช่องเท่ากันเป๊ะ

ลด 20 ฟีเจอร์โดยผลไม่เปลี่ยนแม้แต่ทศนิยมที่ 4 — ยืนยันว่าคอลัมน์พวกนั้นไม่มีข้อมูลจริง

### 3. shap\_top15 — ผลที่ดีที่สุดของงาน

15 ฟีเจอร์    XGB random 0.9803  
79 ฟีเจอร์    XGB random 0.9797

ใช้ฟีเจอร์ 19% ได้ผลเท่ากัน — พิสูจน์ว่า SHAP feature selection ได้ผลจริง

### 4. ตัด Port มีต้นทุนที่ stage 2

no\_const (67, มี Port)    XGB random 0.9797  
no\_port (65, ไม่มี)      XGB random 0.9423      ↓ 0.037

ต่างจากผลของ stage 1 ที่ Port ทำร้าย — สองงานต้องการ Port คนละแบบ (stage 2 แยกชนิด attack: Port มีข้อมูลจริง · stage 1 แยก attack จาก normal: Port เป็น shortcut)

## 5. XGBoost ชนะ random 5 จาก 7 ระดับ

2 ระดับที่แพ้คือ `raw` — แพ้เพราะ XGBoost จัดการฟีเจอร์ที่มีค่าไม่ซ้ำเยอะได้ไม่ดี ( `Flow ID` ไม่ซ้ำ 68% ของแถว) ส่วน DT/RF ชนะเพราะจำ ซึ่งฟังตอนข้าม `testbed`

### ! ยังตอบไม่ได้จากตารางนี้

`cross-env` ทุกช่องอยู่ราว **0.42–0.49** ต่างกันแค่ 0.03–0.07 ซึ่งอาจอยู่ในระดับ `noise` — ยังสรุปไม่ได้ว่าโมเดลไหนดีกว่าตอนข้าม `testbed`

และตัวเลข 0.45 ไม่ได้แปลว่า "ได้ประมาณครึ่ง" ต้องดู `per-class` ถึงจะรู้ว่ามาจากอะไร

## per-class — ตัวเลข 0.45 มาจากอะไร

`macro-F1` เฉลี่ยทุกคลาสเท่ากัน ดังนั้น 0.45 ไม่ได้แปลว่าทุกคลาสได้ **0.45** อาจเป็น 3 คลาสดี + 2 คลาสพัง ก็ได้ — ต้องแยกดูถึงจะตีความได้

(เก็บเฉพาะ XGBoost — ถ้าทำครบ 3 โมเดลจะได้ 9 ตารางซึ่งอ่านไม่ไหว)

```
In [7]: for split in config.SPLITS:
         t = pd.read_csv(config.OUT_DIR / f'02_per_class_{split}.csv', index_col=
         print(f'\n=== split: {split} ===')
         display(t.round(4).style.background_gradient(
             subset=[c for c in t.columns if c != 'support'],
             cmap='RdYlGn', vmin=0, vmax=1))
```

=== split: random ===

|              |       | support | raw      | no_id    | no_const | no_port  | no_dup   | no_expensiv |
|--------------|-------|---------|----------|----------|----------|----------|----------|-------------|
| attack_class |       |         |          |          |          |          |          |             |
| BFA          | 281   |         | 0.996400 | 0.985700 | 0.994700 | 0.906000 | 0.897900 | 0.903500    |
| BOTNET       | 33    |         | 1.000000 | 1.000000 | 0.970600 | 1.000000 | 0.970600 | 0.970600    |
| DDoS         | 24389 |         | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000 | 1.000000    |
| DoS          | 10723 |         | 1.000000 | 0.999700 | 0.999600 | 0.999200 | 0.999100 | 0.999200    |
| Normal       | 13685 |         | 1.000000 | 0.999800 | 0.999600 | 0.999700 | 0.999700 | 0.999700    |
| Probe        | 19626 |         | 0.999900 | 0.999700 | 0.999800 | 0.998400 | 0.998400 | 0.998400    |
| U2R          | 3     |         | 0.857100 | 0.857100 | 1.000000 | 0.857100 | 0.666700 | 0.571400    |
| Web-Attack   | 38    |         | 0.961000 | 1.000000 | 0.973700 | 0.986700 | 0.987000 | 0.974400    |

=== split: ovs2meta ===

|              |       | support  | raw      | no_id    | no_const | no_port  | no_dup  | no_expensiv |
|--------------|-------|----------|----------|----------|----------|----------|---------|-------------|
| attack_class |       |          |          |          |          |          |         |             |
| BFA          | 295   | 0.000000 | 0.022300 | 0.026100 | 0.022300 | 0.024600 | 0.02520 |             |
| DDoS         | 73529 | 0.995100 | 0.996500 | 0.996500 | 0.996500 | 0.996500 | 0.99650 |             |
| DoS          | 1145  | 0.027500 | 0.036900 | 0.032600 | 0.030300 | 0.032000 | 0.03020 |             |
| Normal       | 34212 | 0.839800 | 0.995900 | 0.995000 | 0.993100 | 0.967600 | 0.99270 |             |
| Probe        | 61757 | 0.014400 | 0.352700 | 0.352200 | 0.273500 | 0.273100 | 0.27230 |             |

=== split: meta2ovs ===

|              |       | support  | raw      | no_id    | no_const | no_port  | no_dup  | no_expensiv |
|--------------|-------|----------|----------|----------|----------|----------|---------|-------------|
| attack_class |       |          |          |          |          |          |         |             |
| BFA          | 1110  | 0.010400 | 0.000000 | 0.000000 | 0.004800 | 0.003500 | 0.00110 |             |
| DDoS         | 48413 | 0.000000 | 0.999800 | 0.999800 | 0.999800 | 0.600300 | 0.99980 |             |
| DoS          | 52471 | 0.000000 | 0.001600 | 0.001600 | 0.000300 | 0.000300 | 0.01340 |             |
| Normal       | 34212 | 0.333400 | 0.686200 | 0.648200 | 0.560500 | 0.576300 | 0.62080 |             |
| Probe        | 36372 | 0.000000 | 0.626400 | 0.612700 | 0.702300 | 0.670700 | 0.67960 |             |

## สิ่งที่ per-class เปิดเผย

บน **random** ทุกคลาสเขียวหมด — โมเดลทำได้ดีทุกชนิด attack

บน **cross-env** แยกเป็น 2 กลุ่มชัดเจน:

| คลาส   | cross-env F1 |                           |
|--------|--------------|---------------------------|
| DDoS   | 0.9998       | 🟢 ข้าม testbed ได้สมบูรณ์ |
| Probe  | 0.7116       | 🟡 พอใช้                   |
| Normal | 0.6785       | 🟡 พอใช้                   |
| DoS    | 0.0017       | 🔴 พังสนิท                 |
| BFA    | 0.0000       | 🔴 ทายไม่ถูกเลยสักแถว      |

ตัวเลข 0.45 = 3 คลาสใช้ได้ + 2 คลาสพัง ไม่ใช่ "ได้ประมาณครึ่ง"

## ทำไม 2 คลาสถึงพัง — คนละสาเหตุกัน

เทียบการกระจายของฟีเจอร์ระหว่าง OVS กับ metasploitable ในคลาสเดียวกัน:

| คลาส       | ฟีเจอร์ที่ต่างกันเกิน 1 SD | median diff | cross-env F1  |
|------------|----------------------------|-------------|---------------|
| DDoS       | 2 / 47                     | 0.09        | 0.9998        |
| BFA        | 2 / 58                     | 0.14        | <b>0.0000</b> |
| Probe      | 6 / 65                     | 0.73        | 0.7116        |
| <b>DoS</b> | <b>12 / 63</b>             | <b>0.47</b> | <b>0.0017</b> |

**DDoS** — ข้อมูลแทบเหมือนกันสองที่ (ต่างแค่ 2 ฟีเจอร์) → เข้าได้สมบูรณ์ สมเหตุสมผล

**DoS** — ต่างกันจริง 12 ฟีเจอร์ (OVS 52,471 แถว vs metasploitable 1,145 แถว) น่าจะเป็นคนละเครื่องมือคนละเทคนิค แค่อัดป้ายชื่อเดียวกัน → **distribution shift** แก้ด้วยการปรับโมเดลไม่ได้

**BFA** — ⚠ ต่างกันน้อยที่สุด (2/58) แต่กลับได้ 0 → ขัดกับสมมติฐาน สาเหตุน่าจะเป็น **class imbalance**: ตอน meta2ovs เทรนด้วย BFA แค่ 295 แถว เทียบกับ DDoS 73,529 แถว โมเดลแทบไม่เคยทาย BFA เลย → อันนี้แก้ได้ ด้วย `class_weight` หรือ resampling

## ข้อจำกัดที่ต้องระบุในรายงาน

ตัวเลข cross-env ~0.45 สะท้อนความไม่สอดคล้องของ label ข้าม **testbed** มากกว่าสะท้อนคุณภาพของโมเดล — เพราะ "DoS" ในสองไฟล์อาจเป็นคนละอย่างกัน

จึงไม่ควรใช้ตัวเลขนี้เปรียบเทียบโมเดล แต่ใช้ดู แนวโน้ม ว่าตัดฟีเจอร์แล้วดีขึ้นไหม

## SHAP — โมเดลใช้อะไรตัดสินใจ

XGBoost บอกไม่ได้เองว่าฟีเจอร์ไหนสำคัญ ( `feature_importances_` เป็น Gini importance ซึ่ง bias เข้าหาฟีเจอร์ที่มีค่าหลากหลาย และไม่บอกทิศทาง) จึงต้องใช้ **TreeSHAP**

SHAP คำนวณจาก **train set** เท่านั้น ถ้าคำนวณบน test แล้วเอาไปเลือกฟีเจอร์ = leakage และคำนวณจากโมเดลที่ **จูนพารามิเตอร์แล้ว**

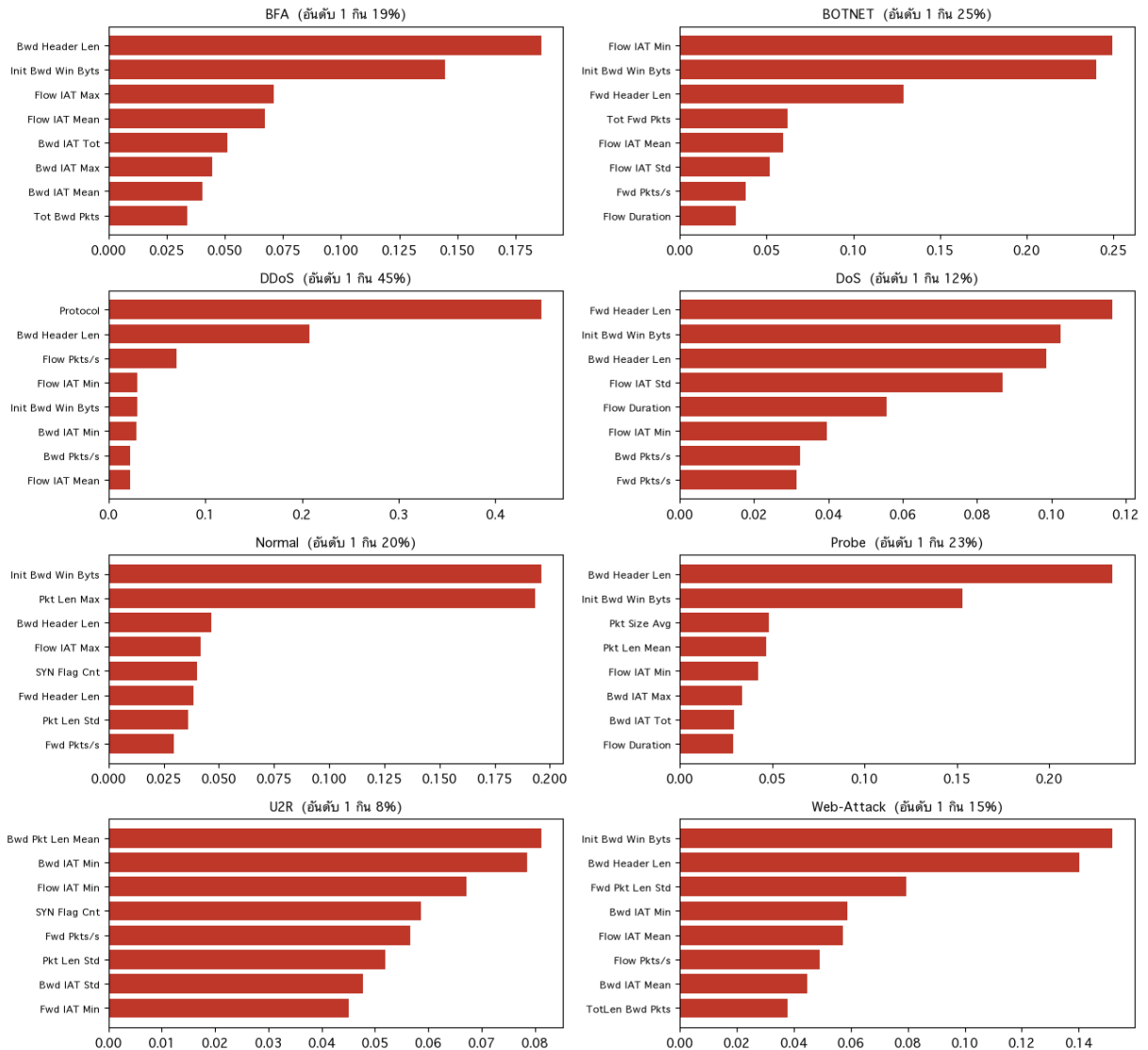
```
In [8]: imp = pd.read_csv(config.OUT_DIR / '03_shap_no_port.csv', index_col=0)

n = len(imp.columns)
fig, axes = plt.subplots((n + 1) // 2, 2, figsize=(13, 3 * ((n + 1) // 2)))
for ax, cls in zip(axes.ravel(), imp.columns):
    top = imp[cls].nlargest(8).sort_values()
    ax.barh(range(len(top)), top.values, color='#c0392b')
    ax.set_yticks(range(len(top)))
    ax.set_yticklabels(top.index, fontsize=8)
    ax.set_title(f'{cls} (อันดับ 1 กิน {top.values[-1]:.0%})', fontsize=10)
```

```

for ax in axes.ravel()[n:]:
    ax.axis('off')
plt.tight_layout()
plt.savefig(config.FIG_DIR / 'shap_per_class.png', dpi=150, bbox_inches='tight')
plt.show()

```



ชื่อฟีเจอร์ในแต่ละกล่องไม่เหมือนกัน = แต่ละ attack ใช้หลักฐานคนละอย่าง

| Attack     | ฟีเจอร์อันดับ 1   | กินความสำคัญ |
|------------|-------------------|--------------|
| DDoS       | Protocol          | 45%          |
| BOTNET     | Flow IAT Min      | 25%          |
| Probe      | Bwd Header Len    | 23%          |
| Normal     | Init Bwd Win Byts | 20%          |
| BFA        | Bwd Header Len    | 19%          |
| Web-Attack | Init Bwd Win Byts | 15%          |
| DoS        | Fwd Header Len    | 12%          |

| Attack | ฟีเจอร์อันดับ 1  | กินความสำคัญ |
|--------|------------------|--------------|
| U2R    | Bwd Pkt Len Mean | 8%           |

## ✔ ตรงกับงานที่ตีพิมพ์

Init Bwd Win Byts ติด Top-5 ของทั้ง 8 คลาส — ตัวเดียวที่ติดครบ

งานที่ benchmark บน InSDN เดียวกัน (2022 ASYU) รายงานว่า Init Bwd Win Byts เป็น ฟีเจอร์สำคัญที่สุดของ InSDN เช่นกัน → pipeline ของเราให้ผลสอดคล้องกับงานที่ตีพิมพ์

## SHAP ยืนยันว่าตัด identifier ถูก

ตัวเลข = อันดับความสำคัญ (1 = สำคัญที่สุด) · - = ถูกตัดไปแล้วในระดับนั้น

```
In [9]: rk = pd.read_csv(config.OUT_DIR / '03_leak_column_rank.csv', index_col=0)
display(rk.fillna('-'))
print('พอตัด identifier ออก Port ได้อันดับขึ้นมาแทนที่')
print('→ โมเดลไม่ได้หันไปเรียนพฤติกรรม แต่หา shortcut ตัวถัดไป')
```

|           | raw  | no_id | no_const |
|-----------|------|-------|----------|
| column    |      |       |          |
| Dst IP    | 5.0  | -     | -        |
| Dst Port  | 7.0  | 3.0   | 1.0      |
| Flow ID   | 2.0  | -     | -        |
| Src IP    | 10.0 | -     | -        |
| Src Port  | 9.0  | 4.0   | 6.0      |
| Timestamp | 1.0  | -     | -        |

พอตัด identifier ออก Port ได้อันดับขึ้นมาแทนที่  
→ โมเดลไม่ได้หันไปเรียนพฤติกรรม แต่หา shortcut ตัวถัดไป

## แล้วจะรวมฟีเจอร์ยังไง

แต่ละคลาสได้ฟีเจอร์ไม่เหมือนกัน แต่เราต้องการ ชุดเดียว ไปเทรน

| วิธี         | นิยาม                             | ได้กี่ตัว |
|--------------|-----------------------------------|-----------|
| union        | ติด Top-15 ของคลาสไหนก็เอา        | 39        |
| intersection | ต้องติด Top-15 ของ ทุกคลาส        | 1         |
| mean         | เฉลี่ยข้ามคลาสก่อน แล้วตัด Top-15 | 15        |

| วิธี    | นิยาม                                 | ได้กี่ตัว |
|---------|---------------------------------------|-----------|
| dynamic | แต่ละคลาสเก็บจนสะสมถึง 90% แล้ว union | 45        |

**intersection** เหลือตัวเดียวคือ **Flow IAT Min** — ซ้ำเกินไปจนใช้ไม่ได้

**union** 39 ตัวจาก 65 — ยังเยอะ แทบไม่ต่างจากไม่กรอง

**dynamic** เลือกจำนวนต่างกันตามคลาสจริง: DDoS ใช้ 11 · BOTNET 10 · BFA 21 · Normal 23 · U2R 26 · Probe 27 · **DoS 28**

→ คลาสที่ตรวจจับยากต้องใช้ฟีเจอร์มากกว่า สอดคล้องกับผล cross-env ที่ DoS พังที่สุด

---

## ขั้นถัดไป (ยังไม่ใช่เนื้อหาของ present นี้)

ทดลองเบื้องต้น: เลือก 15 ฟีเจอร์ ด้วยวิธี **mean** → macro-F1 **0.9775** เทียบกับใช้ 67 ฟีเจอร์ = 0.9922 → ต่างกัน **0.015** โดยใช้ฟีเจอร์ **22%**

ยังต้องทดลองต่อว่า K เท่าไหร่ดีที่สุด และวิธีรวมแบบไหนดีกว่ากัน